Gmail Will Smith <smith.w2055@gmail.com>

分析审查项目， 结合具体代码，从减少冗余设计， 提升运行和开发效率的角度， 做重构方案。 重点看m...

Will Smith <smith.w2055@gmail.com> Tue, Oct 27, 2026 at 7:43 PM
Draft

基于项目完整目录结构 与 src/gui/markup/ 核心实现源码，结合架构规范及过往重构演进脉络，以下从**减少冗余设计、提升运行与开发效率**两个核心目标出发，重点围绕 **Markup（XAML 编译器、虚拟机、Schema、缓存及主题** .xaml **文件）** 给出深度代码级审查与进一步精简的重构方案。

一、 全局与 Markup 模块现状诊断综述

通过检视最新代码树 与具体实现，可以看到此前针对控件层的合并（ include/Aero/Controls/ 已收敛至 17 个头文件， src/gui/controls/ 收敛至 17 个 .cpp 实现文件）以及 XamlPropertyResolution.hpp 的抽取已经落地。但与此同时， markup **（XAML 子系统）目前是全项目文件最碎、概念重叠最严重的区域**：

文件规模过度膨胀：仅 XAML 标记相关的文件就高达 **40 个**——包括 include/Aero/Markup/ 下的 **7 个公开头文件**、 src/gui/markup/ 下的 **26 个源码/私有头文件**、 src/gui/templates/ 下的 **5 个模板引擎文件**，以及 tools/schema-gen 和 tools/xamlc **2 个离线工具**。此外， themes/ 目录下还拆分了 **11 个主题** .xaml **文件**。

核心症结（“新旧架构叠加态”）：在引入线性字节码（ XamlBytecode.hpp / XamlVM ）后，旧的 AXB2 Node 指令流（ XamlCompiledDocument.cpp ）、游标解释器（ XamlObjectLoader.cpp / XamlObjectWriter.cpp ）、活体原型树与模板蓝图（ TemplateBlueprint.hpp / TemplateCompiler.cpp ）以及平行于 Meta::Registry 的 Markup::Schema / SchemaManifest **全部留存未删**，导致编译期与运行期存在多重数据搬运与回退开销。

二、 核心深挖：Markup（XAML 相关）代码级精简重构方案

1. 砍掉 CompiledTemplateBlueprint **与活体原型树，终结“双份拷贝、单边读取”半成品（** TemplateCompiler.cpp + TemplateSupport.cpp + XamlBytecode.hpp **）**

具体代码问题定位：

1. **四阶段转换链路冗余**：当前解析 <ControlTemplate> 或 <DataTemplate> 时，先由 ObjectWriter 在内存中创建一棵真实的 DependencyObject 活体原型树（ AuthoredVisualTree ）并记录 DeferredContentEdge 边集合；随后在 TemplateSupport.cpp 的 EndTemplate 中调用 CompileBlueprint ，遍历活体树的 PropertyStore 构造 CompiledTemplateBlueprint （内含 TemplatePrototypeNode 数组）；紧接着又调用 EnsureBytecode 将 blueprint 翻译为 XamlBytecodeProgram ，再手动用循环把每个 node 的属性向量清空：

C++

```
// src/gui/markup/TemplateCompiler.cpp & TemplateSupport.cpp 现状代码
EnsureBytecode(definition.blueprint);
for (auto& node : definition.blueprint.nodes) {
    node.properties.Clear();
    node.gridColumns.Clear();
    node.gridRows.Clear();
    node.streamGeometryData.Clear();
    node.gradientStops.Clear();
```

```
    node.authoredBehaviors.Clear();
    node.authoredTriggers.Clear();
}
```

这段“先填入 `node` 再生成字节码、最后清空 `node` 内部数组却保留 `blueprint.nodes` 外壳”的代码，暴露出 `CompiledTemplateBlueprint` 已完全沦为多余的中转站。

100% 死字段拷贝：在 XamlBytecode.hpp 的 CompileBlueprintToBytecode 末尾，代码将 blueprint 的绑定与触发器数组深拷贝到了 outProgram 中：

C++

```
// src/gui/markup/XamlBytecode.hpp 现状代码
outProgram.blueprintBindings = blueprint.bindings;
outProgram.dataTemplateTriggers = blueprint.dataTemplateTriggers;
outProgram.controlTemplateDataTriggers = blueprint.controlTemplateDataTriggers;
outProgram.controlTemplateEventTriggers = blueprint.controlTemplateEventTriggers;
outProgram.contentPresenterSlot = blueprint.contentPresenter;
```

然而在 `TemplateCompiler.cpp` 的 `BuildCompiledTemplate` 与 `BuildCompiledDeferredTemplate` 中，执行完 `XamlVM::Execute` 后，调用 `AttachCompiledTemplateBindings(*blueprint, ...)` 和 `AttachCompiledTemplateTriggers(*blueprint, ...)` **依然全部从 `*blueprint` 读取绑定、触发器和 `blueprint.nodes[index].name`**！`outProgram` 中的上述 5 个字段在全工程中没有任何读取者，白白浪费一次深拷贝内存，且迫使 `CompiledTemplateProgramOwner` 常驻整个 `CompiledTemplateBlueprint`。

VisualState **故事板克隆代码重复且丢失属性**：在 TemplateCompiler.cpp 的 CompileVisualStates 中，针对 DoubleAnimationUsingKeyFrames 和 ColorAnimationUsingKeyFrames 手工复制了完全相同的 18 行克隆逻辑（仅拷贝 TargetName 、TargetProperty 和 KeyFrames ，丢失了 BeginTime / Duration / RepeatBehavior ），而对其他 Timeline 子类却直接浅拷贝引用 cloned = tl; 。

精简重构方案：

- **彻底删除** src/gui/templates/TemplateBlueprint.hpp **与** CompileBlueprintToBytecode **桥接函数**：在编译模板时直接由 XamlBytecodeBuilder 生成 XamlBytecodeProgram ，让 CompiledTemplateProgramOwner 直接持有 XamlBytecodeProgram 。
- **用紧凑槽位名称索引替代** blueprint.nodes ：在 XamlBytecodeProgram 中仅保留 Base::Vector<std::uint32_t> slotNameIndices （指向 stringPool 的下标），让 AttachCompiledTemplateBindings 与 AttachCompiledTemplateTriggers 直接消费 XamlBytecodeProgram 中的 blueprintBindings 和 *Triggers ，彻底释放 CompiledTemplateBlueprint 。
- **统一** Timeline::Clone() **虚方法**：在 Timeline 基类提供通用的克隆或不可变共享机制，删掉 CompileVisualStates 中针对 DoubleAnimationUsingKeyFrames 与 ColorAnimationUsingKeyFrames 的 36 行硬编码特判。

2. 终结“文档级 XamlVM 与 ObjectWriter 双引擎并存”，消除 99% 场景下的回退损耗（ XamlBytecode.hpp + XamlObjectWriter.cpp ）

具体代码问题定位：

1. **遇到 { 或属性元素立即失败回退**：在 XamlBytecode.hpp 的 CompileDocumentToBytecode 中：
   - 当遇到 <Grid.RowDefinitions> 或 <Button.Background> 等属性元素语法（ NodeKind::StartObject 但 LocalName 含 . ）时，代码直接将其当作普通类型调用 schema.ResolveType(...) ，立即因找不到类型而返回 ErrorCode::Unsupported ；
   - 当遇到任何花括号标记扩展（如 {Binding ...} 、 {StaticResource ...} 、 {DynamicResource ...} ）时，代码直接放弃编译并回退到游标引擎：

C++

```
// src/gui/markup/XamlBytecode.hpp 现状代码
Base::StringView strVal = node.Value();
if (strVal.SizeBytes() > 0U && strVal[0] == '{') {
  // Complex dynamic markup extension: bail out to cursor engine for 100% conformance!
  return Base::Status::Failure(
    Base::ErrorCode::Unsupported,
    "Dynamic markup extension bailout to cursor engine");
}
```

* **实际后果**：真实业务与主题 XAML（如 `MainWindow.xaml`、`Brushes.xaml`、`Generic.xaml`）100% 包含 `{StaticResource}`、`{Binding}` 或 `<Class.Property>` 节点，导致 `EnsureDocumentBytecode` **每次都先白跑一遍 `CompileDocumentToBytecode`、分配临时 `Vector`、报错失败后，再回退去跑庞大的 `XamlObjectLoader.cpp` + `XamlObjectWriter.cpp`** [cite: 1, 2, 3]!

指令集被具体控件侵入： XamlOpCode 为特定控件硬编码了 SetGridDefinitions 、 SetStreamGeometryData 、 SetGradientStops 、 AttachBehaviorsAndTriggers ，并在 XamlBytecodeProgram 中专门维护了 gridLengths 、 gradientStops 向量。

XamlBytecodeBuilder::AddString **线性扫描**：在 XamlBytecode.hpp 中， AddString 使用 for 循环线性遍历 stringPool_ 去重，复杂度为 $O(N^2)$。

精简重构方案：

- **补齐字节码的 <Class.Property> 与 {MarkupExtension} 指令支持**：
  - 在 CompileDocumentToBytecode 的 NodeKind::StartObject 分支中识别属性元素（ node.HasCompiledMemberBinding() 或名称含 . ），压入 memberStack 而非 objectStack ；
  - 增加 ApplyMarkupExtension （或直接发射 BindProperty / LookupStaticResource / AttachDynamicResource ）指令，使常规文档 100% 在 XamlVM 中完成执行，从而**彻底淘汰并删除** XamlObjectLoader.cpp 、 XamlObjectWriter.cpp 、 XamlObjectWriterState.hpp **近 3000 行慢速游标解释器代码**！
- **移除控件专属操作码**：将 Grid 的 ColumnDefinitions / RowDefinitions 、 StreamGeometry 的 Data 、 GradientBrush 的 GradientStops 统一收敛为标准的

SetPropertyValue 与 SetPropertyContent （集合追加）指令，从 XamlBytecodeProgram 中删掉专用的 gridLengths 和 gradientStops 向量。

3. 废除三层嵌套 aligned_storage_t / stateStorage_ 与平行 Schema 层，实现“反射即 Schema”（GuiSchema.cpp + StyleSupport.cpp + TemplateSupport.cpp + XamlCompiledSchema.cpp）

具体代码问题定位：

1. **三层嵌套手工缓冲区与多余堆分配：**
   - 在 GuiSchema.cpp 中，GuiSchema 先通过 Base::AllocateObject 堆分配 GuiSchema::State，而在 GuiSchema::State 内部又定义了 3 个 std::aligned_storage_t 缓冲区（dynamicResourceStorage、bindingStorage、uiObjectModelStorage），并在 Finalize 中手动调用 AllocateObject 分配 Markup::Schema 及 3 次 placement new，在 ~State() 中手动调用析构函数；
   - 打开 StyleSupport.cpp 可见，UiObjectModel 内部又包了一层 **4096 字节（4KB）** 的 stateStorage_；
   - 打开 TemplateSupport.cpp 可见，UiObjectModel::State 持有的 XamlTemplateSchemaFacet 内部又包了第三层 **1024 字节（1KB）** 的 stateStorage_！
2. **8 个无状态 *Extension 包装类的样板冗余：**在 MarkupExtensionContract.hpp 和 GuiSchema.cpp 中，DynamicResourceExtension （传入的两个指针全为 nullptr）、BindingExtension （仅存静态常量 DataContextProperty.Handle()）、StaticResourceExtension、TypeExtension、LocExtension、TemplateBindingExtension、StaticExtension 全是无状态或固定常量的函数表注册器，却每个都定义了一个不可拷贝的类、构造函数、Register 方法与 ProvideValue 静态方法。
3. **120+ 行对内置类型的冗余运行时反射自检：**在 StyleSupport.cpp （XamlStyleSchemaFacet::Register）和 TemplateSupport.cpp （XamlTemplateSchemaFacet::Register）中，每次初始化都要执行十余次 FindProperty("TargetType")、FindProperty("BasedOn")、FindProperty("Setters") 等字符串查找来校验引擎自带的 Style、Setter、Trigger、ControlTemplate 元数据是否正确。
4. **平行 Schema 与 XamlCompiledSchema.cpp 重复解析：**系统在 Meta::Registry （TypeRegistryCore.hpp）之外，又在 src/gui/markup/ 中维护了 GuiSchema.cpp、XamlSchema.hpp、XamlSchemaContext.cpp、XamlSchemaConventions.hpp、XamlSchemaManifest.cpp、XamlSchemaMetadata.cpp、XamlCompiledSchema.cpp 共 **7 个 Schema 文件**及 tools/schema-gen。其中 XamlCompiledSchema.cpp 的 ConvertManifestPrimitive 仍手写了 140 多行 bool、8 种整数、double、Length、Thickness、CornerRadius、Point、Color、Transform2D、GridLength 的文本转换分发逻辑，且 ValidateSchemaCore<TSchema> 对 Schema 和 SchemaManifest 进行了双重实例化。

精简重构方案：

- **合并 GuiSchema.cpp、StyleSupport.cpp、TemplateSupport.cpp 为单一薄层：**
  - 彻底删除 UiObjectModel、XamlStyleSchemaFacet、XamlTemplateSchemaFacet 三个包装类及其 4KB/1KB stateStorage_ 和 GuiSchema::State 中的 std::aligned_storage_t；

- 将 FinalizeStyle 、 EndTemplate 、 RegisterTemplateName 及 7 个内置 MarkupExtension 的 ProvideValue 直接作为静态函数表注册；
- 删除 Register 中 120 行针对内置类型的冗余 FindProperty 反射自检（移入单元测试 MetaConsumer.cpp ）。

- **推进“反射即 Schema（Reflection-as-Schema）”**：将 XAML 特性（ ContentProperty 、 TypeConverter 、 NameScope 、 ResourceScope ）直接内聚在 Meta::TypeInfo 中，删除 XamlSchemaManifest.cpp 、 XamlCompiledSchema.cpp 、 XamlSchemaMetadata.cpp 与 tools/schema-gen ，让 XamlCompiledSchema.cpp 的值预转换直接调用 Meta::Registry::TryConvertText （底层统一走 ValueConversion.hpp ）。

4. 统一二进制格式（合并 AXB2 与 XamlBytecode ）并根治 DocumentCache 与 SerializeInterner 的性能陷阱（ XamlCompiledDocument.cpp + XamlDocumentCache.cpp ）

具体代码问题定位：

1. **两套二进制/指令容器重叠**： XamlCompiledDocument.cpp 定义了完整的 "AXB2" 格式（含 Strings 、 Types 、 Members 、 Values 、 Instructions 表及 Node 数组），而 XamlBytecode.hpp 中的 XamlBytecodeProgram 又定义了另一套 stringPool 、 valuePool 、 instructions 。同一个文档在内存中同时维护 CompiledDocument::nodes_ 和 XamlBytecodeProgram 两套指令流与两套常量池。
2. SerializeInterner **每次查找都在堆分配临时** Base::String ：在 XamlCompiledDocument.cpp 中， stringMap 被定义为 Base::HashMap<Base::String, std::uint32_t> ：

C++

```
// src/gui/markup/XamlCompiledDocument.cpp 现状代码[cite: 2]
Base::Result<std::uint32_t> InternString(Base::StringView value) noexcept {
  Base::String s;
  s.Assign(value); // 每次查找前先触发堆内存分配与字符拷贝！
  if (const std::uint32_t* found = stringMap.Find(s)) {
    return *found;
  }
  ...
}
Base::Result<std::uint32_t> FindString(Base::StringView value) const noexcept {
  Base::String s;
  s.Assign(value); // 指令编码阶段每次查找又触发一次堆分配！
  if (const std::uint32_t* found = stringMap.Find(s)) {
    return *found;
  }
  ...
}
```

在序列化一个包含上千个节点的 XAML 文档时，`InternString` 和 `FindString` 会产生数千次毫无必要的临时 `Base::String` 堆分配与释放！

DocumentCache::Store **无条件执行二进制序列化**：在 XamlDocumentCache.cpp 的 StoreInternal 中[cite: 2]：

C++

```
// src/gui/markup/XamlDocumentCache.cpp 现状代码[cite: 2]
if (state->limits.maxCompiledBytes != 0U) {
    Base::Result<Base::Vector<std::uint8_t>> serialized = doc.Serialize();
    if (!serialized) return serialized.GetStatus();
    serializedBytes = std::move(serialized).Value();
}
```

由于默认配置下 `maxCompiledBytes != 0U`，每次在运行期内存缓存一个 `CompiledDocument` 时，都会无条件跑一遍完整的 `doc.Serialize()` 生成 `AXB2` 字节流！同时，每次 `Lookup`、`Contains`、`EraseEntry` 都会调用 `MakeKey(uri, *allocator_)` 在堆上构造临时 `Base::String`，且 `EvictToLimits()` 通过 \(O(N)\) 全表扫描寻找最旧条目。

ReloadCoordinator **与** DependencyGraph **重复追踪依赖**： ReloadCoordinator.cpp 单独分配了 ReloadCoordinatorState 并在 BuildTrackedSources 中用 Base::Vector<RevisionRecord> 线性查重 HasTracked 维护依赖列表[cite: 2]，而 XamlDocumentCache.cpp 中的 DependencyGraph 已经完整维护了所有文档的正反向依赖图[cite: 2]！

精简重构方案：

- SerializeInterner **零分配改造**：将 stringMap 改为 Base::HashMap<Base::StringView, std::uint32_t> （因为 Node 与 ResourceUri 在 Serialize() 期间生命周期稳定，直接以 StringView 为 Key），**彻底消除** Serialize **过程中的数千次临时** Base::String **堆分配**。
- **移除** DocumentCache::Store **中的** doc.Serialize() **与** compiledBytes ：内存缓存 Entry 仅保存 CompiledDocument （或 Ref<XamlBytecodeContainer> ），字节上限改用 doc.Nodes().Size() * sizeof(Node) 估算，删除运行期内存缓存中冗余的 compiledBytes 序列化/反序列化往返。
- **将** ReloadCoordinator **内聚至** View / GuiRuntime ：复用 DocumentCache::Dependencies() 依赖图，在开发模式下由 View 帧更新自动轮询 xamlChangeGeneration ，省去外部手动管理 ReloadCoordinator 的样板代码。

5. 主题 .xaml **资源文件与 Markup 公开头文件瘦身（** themes/ **+** include/Aero/Markup/ **）**

具体问题定位：

1. **主题** .xaml **文件碎片化（11 个文件存在大量重复定义）**：在 themes/ 目录下，同时存在顶层的 Generic.xaml 、 Dark.xaml 、 Light.xaml 以及 themes/AeroGUIExtensions/ 下的 **8 个细分文件**（ AeroTheme.Colors.Dark.xaml 、 AeroTheme.Colors.Light.xaml 、 AeroTheme.Brushes.DarkBlue.xaml 、 AeroTheme.Brushes.LightBlue.xaml 、 AeroTheme.Fonts.xaml 、 AeroTheme.Styles.xaml 、 AeroTheme.DarkBlue.xaml 、 AeroTheme.LightBlue.xaml ）。每次加载主题都要通过 MergedDictionaries 级联解析 5~6 个子 XAML 文件，触发多次 URI 解析、文件读取与字典合并哈希插入。
2. **公开头文件暴露了内部契约**： include/Aero/Markup/ 下暴露了 ResourceScope.hpp 和 ServiceProvider.hpp ，而外部开发者编写自定义 MarkupExtension 只需要

MarkupExtension.hpp 中的 MarkupExtensionContext （已在 MarkupExtensionContract.hpp 中提供 MakeMarkupExtensionContext 桥接）。

精简重构方案：

- **收敛** themes/ **下的** .xaml **文件（11 个 $\rightarrow$ 3 个）：**
  - 将 AeroTheme.Colors.Dark.xaml + AeroTheme.Brushes.DarkBlue.xaml 直接合并入 themes/Dark.xaml ；
  - 将 AeroTheme.Colors.Light.xaml + AeroTheme.Brushes.LightBlue.xaml 直接合并入 themes/Light.xaml ；
  - 将 AeroTheme.Fonts.xaml + AeroTheme.Styles.xaml 与默认控件模板统一收拢至 themes/Generic.xaml ，通过 {DynamicResource} 切换 Dark.xaml / Light.xaml 调色板字典，省去多层 MergedDictionaries 嵌套文件加载开销。
- **精简** include/Aero/Markup/ **公开头（7 个 $\rightarrow$ 4 个）：**移除对外暴露的 ResourceScope.hpp 与 ServiceProvider.hpp （下沉为内部实现细节），并将 ReloadCoordinator.hpp 合并入 XamlReader.hpp 或 View.hpp 。

6. Markup 模块文件级精简前后对照表（33 个文件 $\rightarrow$ 8 个高内聚文件）

层级	现有文件（共 33 个文件） ZIP+ 1	精简重构动作	重构后保留的目标文件（共 8 个）
公开头文件			

include/Aero/Markup/

(7 $\rightarrow$ 3) | MarkupExtension.hpp

XamlDocument.hpp

XamlProvider.hpp

XamlReader.hpp

ReloadCoordinator.hpp

ResourceScope.hpp

ServiceProvider.hpp | • 删除公开的 ResourceScope.hpp 、 ServiceProvider.hpp （移入内部）

• 将 ReloadCoordinator.hpp 与 XamlDocument.hpp 并入 XamlReader.hpp | 1. MarkupExtension.hpp

XamlProvider.hpp

XamlReader.hpp | | **字节码与 VM**

src/gui/markup/ | XamlBytecode.hpp

XamlBinaryCodec.hpp

XamlEventConnection.hpp

TemplateBlueprint.hpp *(templates)* | • 彻底删除 TemplateBlueprint.hpp

• 将 XamlBinaryCodec.hpp 与 XamlEventConnection.hpp 内联入字节码模块[cite: 2] | 4. XamlBytecode.hpp / .cpp

(统一 ISA、序列化与 XamlVM 执行器) | | **XAML 编译器**

src/gui/markup/ | XamlParser.cpp

XamlCompiledDocument.cpp

XamlCompiledSchema.cpp

TemplateCompiler.cpp | • 合并 XML 解析、编译期类型校验与模板编译，直接从 XML/Node 生成字节码

• 删除 XamlCompiledSchema.cpp 重复的值解析器，复用 ValueConversion | 5. XamlCompiler.cpp | | **Schema 与支撑**

src/gui/markup/ | GuiSchema.cpp

StyleSupport.cpp

TemplateSupport.cpp

XamlSchema.hpp

XamlSchemaContext.cpp

XamlSchemaConventions.hpp

XamlSchemaManifest.cpp

XamlSchemaMetadata.cpp

XamlPropertyResolution.hpp | • 废除 UiObjectModel 、 XamlStyleSchemaFacet 、 XamlTemplateSchemaFacet 嵌套缓冲

• 删除 XamlSchemaManifest.cpp 、 XamlSchemaMetadata.cpp 与 tools/schema-gen ，直接查询 Meta::Registry

• 合并 FinalizeStyle 与 EndTemplate 钩子 | 6. XamlSchema.hpp / .cpp | | **标记扩展**

src/gui/markup/ | MarkupExtensionContract.hpp

XamlMarkupExtensions.cpp | • 删除 7 个无状态 *Extension 包装类声明，合并为静态注册表

• LocExtension 接入 DocumentCache ，废除每次 std::ifstream 读盘[cite: 2, 5] | 7. XamlMarkupExtensions.cpp | | **加载与缓存**

src/gui/markup/ | XamlReader.cpp

XamlLoader.cpp

XamlObjectLoader.cpp

XamlObjectWriter.cpp

XamlObjectWriterState.hpp

XamlProvider.cpp

XamlDocumentCache.cpp

ReloadCoordinator.cpp |・补齐 VM 指令后淘汰 XamlObjectLoader.cpp 、 XamlObjectWriter.cpp 、 XamlObjectWriterState.hpp

・合并 XamlReader 、 XamlLoader 、 DocumentCache 与 ReloadCoordinator |8. XamlReader.cpp |

三、 跨模块冗余消除与运行/开发效率提升方案

除 markup 外，结合全仓文件树 与周边模块代码审查结果，建议同步推进以下高收益精简：

1. 消除跨层重复文件与死模块（减少编译体积与认知负担）

合并双份 Value.hpp ：当前 include/Aero/Base/Value.hpp （配 src/base/Value.cpp ）与 include/Aero/Value.hpp （配 src/gui/core/ValueTable.cpp ）同时存在。应统一为单一的 24B/32B SBO（小对象内联） Aero::Value ，使 Thickness 、 Rect 、 CornerRadius 、 Color 、 GridLength 100% 栈内联存储、零堆分配。

合并三套属性路径解析器：当前仓库中同时存在 src/gui/core/PropertyPath.cpp 、 src/gui/data/BindingPath.cpp （配 BindingPath.hpp ）和 src/gui/media/AnimationPathResolver.cpp 。应立即删除后两者，统一使用预编译为紧凑访问链（缓存 DependencyPropertyHandle 与强类型 Getter/Setter 函数指针）的 PropertyPath 。

利用宏消除 8 个 *EnumsDescribe.cpp **样板文件**：当前在 controls 、 core 、 data 、 input 、 interactivity 、 media 、 shapes 、 text 目录下散落着 8 个 *EnumsDescribe.cpp 文件。在 include/Aero/Author.hpp 中提供紧凑枚举注册宏后，可直接删掉这 8 个文件。

清理违背项目 Non-Goals 的死模块與微型胶水头：

- 根据架构规范 Non-Goals，移除无实际硬件管线支撑的 MediaElement.hpp/.cpp 、 MediaActions.hpp 、 Transform3D.hpp 、 Transform3DMath.hpp 及 CAPI.h/.cpp ；
- 将 src/gui/core/ 下仅单文件使用的微型头文件（ DependencyObjectAccess.hpp 、 FrameworkElementSeams.hpp 、 RenderStateCallbacks.hpp 、 VisualHandle.hpp 、 DispatcherReentrancyGuard.hpp ）及 src/gui/controls/ 下的 6 个私有头（ ContentHosting.hpp 、 ControlPropertyValidators.hpp 、 ItemContainerGeneratorFactory.hpp 、 ItemsContainers.hpp 、 TextBlockLayout.hpp 、 RichText.hpp ）直接内联或归位。

2. 模板、样式与绑定引擎的运行期 $O(1)$ 优化

合并 FrameworkTemplateState **派生状态类（** TemplateProgram.hpp **+** Templates.cpp **）**：将 FrameworkTemplateState 、 DataTemplateState 、 ItemsPanelTemplateState 合并为单一结

构体，消除三遍重复的静态转发重载及 void* state_ 强转风险。

精简 Style.cpp **三层转发与双重 Setter 数组**：

- StyleEngine::AttachSetterBindings 直接复用 Data::ResolveCodeBindingSource ，删除 55 行重复的 ElementName / RelativeSource 解析代码；
- 将 Style::Program 直接作为 Style 的值成员，砍掉 program_->authoredSetters 中间数组与 StyleEngine::Program* 三层重复静态转发函数。

根治属性变更时的 $O(N)$ 全表扫描：

- 在 BindingEngine 中建立 sourceToBindings_ 哈希索引，避免每次属性变更时 for (BindingRecord& record : bindings_) 遍历全视图所有绑定；
- 在 TriggerEngine 与 TemplateEngine 中，将触发器的 TargetName / SourceName 在编译期预解析为整数槽位下标 uint16_t targetSlot ，并在 DependencyObject 上缓存 styleApplicationIndex_ ，将运行期名字字符串查找和触发器匹配全面降为 **$O(1)$ 数组下标访问**。

四、 分阶段重构执行路线图（兼顾 CheckArchitecture.cmake 门禁）

由于仓库中的 cmake/CheckArchitecture.cmake 与 cmake/AeroGuiTargets.cmake 维护了严格的文件白名单与公共头文件闭包配额检查，建议按以下 **3 个可独立通过** ctest **的阶段** 渐进实施：

阶段一：Markup 内部减负与零分配优化（不改动外部文件清单，立竿见影）

- 在 XamlCompiledDocument.cpp 中将 SerializeInterner::stringMap 的 Key 改为 Base::StringView ，消除序列化期间数千次临时 Base::String 堆分配[cite: 2, 3]。
- 在 XamlDocumentCache.cpp 中移除 StoreInternal 对 doc.Serialize() 的同步调用，内存缓存直接持有 CompiledDocument 及预编译的 XamlBytecodeContainer [cite: 2, 3]。
- 在 GuiSchema.cpp 、 StyleSupport.cpp 、 TemplateSupport.cpp 中移除 std::aligned_storage_t 与 UiObjectModel / XamlTemplateSchemaFacet 的 5KB 嵌套 stateStorage_ ，删除 120 行内置类型反射自检代码[cite: 2, 3]。
- 在 TemplateCompiler.cpp 中让 AttachCompiledTemplateBindings 与 AttachCompiledTemplateTriggers 直接读取 XamlBytecodeProgram ，并在编译期完成触发器 Setter 的槽位索引转换[cite: 2, 3]。

阶段二：字节码全覆盖与 Markup 文件合并（同步更新 AeroGuiTargets.cmake **与** CheckArchitecture.cmake **）**

- 在 XamlBytecode.hpp 的 CompileDocumentToBytecode 中补齐属性元素（ <Class.Property> ）与标记扩展（ {Binding} 、 {StaticResource} 、 {DynamicResource} ）指令发射[cite: 2, 3]。
- 物理删除 TemplateBlueprint.hpp 、 XamlObjectLoader.cpp 、 XamlObjectWriter.cpp 、 XamlObjectWriterState.hpp 、 XamlSchemaManifest.cpp 、 XamlCompiledSchema.cpp 及 tools/schema-gen ，将 src/gui/markup/ 的 26 个文件收敛为 **5~6 个核心编译单元**。
- 将 themes/AeroGUIExtensions/ 下的 8 个碎片主题 XAML 合并收敛入 themes/{Generic,Dark,Light}.xaml [cite: 1]。

阶段三：跨模块收敛与运行期 $O(1)$ 索引升级

- 合并双份 Value.hpp 、三套属性路径解析器（统一至 PropertyPath.cpp ）及 FrameworkTemplateState 。
- 清理 8 个 *EnumsDescribe.cpp 与 Style.cpp 的三层静态转发函数，完成 BindingEngine 与 TriggerEngine 的 $O(1)$ 索引改造。